Tryton

Tryton to superkomputer o architekturze klastrowej o następujących parametrach:

Procesory:	Intel® Xeon® Processor E5 v3 @ 2,3 GHz, 12-core (Haswell)
Akceleratory:	Nvidia Tesla, Intel Xeon Phi, AMD FirePro
Pamięć:	128/256 GB RAM DDR4 na serwer
Sieć:	InfiniBand FDR 56 Gb/s, topologia fat tree, przełączniki Mellanox
Razem:	1483 serwery, 2966 procesorów, 35592 rdzeni, 48 akceleratorów, 202 TB RAM
Szafy:	40 szt.
Moc obliczeniowa:	1,37 PFLOPS

Superkomputer zbudowany jest z podzespołów zakupionych z funduszy projektów CD NIWA i PLGrid Plus:

Część projektu CD NIWA

Platforma:	HP Apollo 6000 z serwerami HP ProLiant XL230a Gen9
Procesory:	Intel® Xeon® Processor E5 v3 @ 2,3 GHz, 12-core (Haswell)
Akceleratory:	Nvidia Tesla, Intel Xeon Phi, AMD FirePro
Pamięć:	128/256 GB RAM DDR4 na serwer
Sieć:	InfiniBand FDR 56 Gb/s, topologia fat tree, przełączniki Mellanox
Razem:	1308 serwerów, 2616 procesorów, 31392 rdzeni, 48 akceleratorów, 180 TB RAM
Szafy:	34 szt.
Moc obliczeniowa:	1,216 PFLOPS

Część projektu PLGrid Plus

Platforma:	Serwery Actina SOLAR 820 S6
Procesory:	Intel® Xeon® Processor E5 v3 @ 2,3 GHz, 12-core (Haswell)
Pamięć:	128 GB RAM DD4 na serwer
Sieć:	InfiniBand FDR 56 Gb/s, topologia fat tree, przełączniki Mellanox
Razem:	175 serwerów, 350 procesorów, 4200 rdzeni, 22 TB RAM
Szafy:	6 szt.
Moc obliczeniowa:	0,154 PFLOPS

Aplikacje systemowe

- Intel MPI
- OpenMPI
- Oracle
- MPICH
- MVAPICH
- PBS
- SLURM
- Tivoli Storage Manager
- vSMP

In superscalar designs, the number of execution units is invisible to the instruction set. Each instruction encodes only one operation. For most superscalar designs, the instruction width is 32 bits or fewer.

In contrast, one **VLIW** instruction encodes multiple operations; specifically, one instruction encodes at least one operation for each execution unit of the device. For example, if a VLIW device has five execution units, then a VLIW instruction for that device would have five operation fields, each field specifying what operation should be done on that corresponding execution unit. To accommodate these operation fields, VLIW instructions are usually at least 64 bits wide, and on some architectures are much wider.

In computer architecture, a **branch target predictor** is the part of a processor that predicts the target of a taken conditional branch or an unconditional branch instruction before the target of the branch instruction is computed by the execution unit of the processor.

Branch target prediction is not the same as branch prediction which attempts to guess whether a conditional branch will be taken or not-taken (i.e., sequential).

In more parallel processor designs, as the instruction cache latency grows longer and the fetch width grows wider, branch target extraction becomes a bottleneck. The recurrence is:

- Instruction cache fetches block of instructions
- Instructions in block are scanned to identify branches
- First predicted taken branch is identified
- Target of that branch is computed
- Instruction fetch restarts at branch target

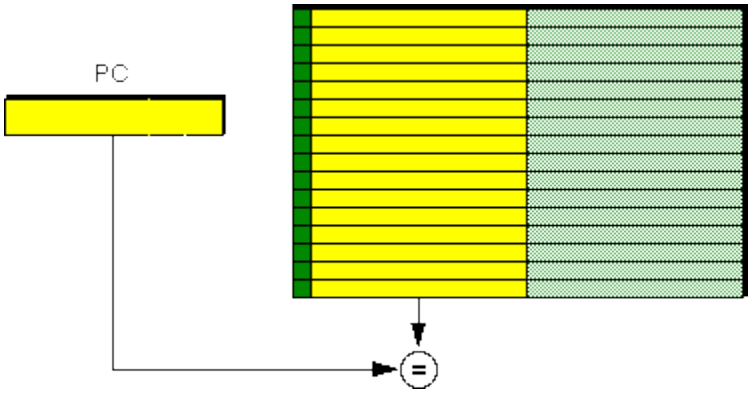
In machines where this recurrence takes two cycles, the machine loses one full cycle of fetch after every predicted taken branch. As predicted branches happen every 10 instructions or so, this can force a substantial drop in fetch bandwidth. Some machines with longer instruction cache latencies would have an even larger loss. To ameliorate the loss, some machines implement branch target prediction: given the address of a branch, they predict the target of that branch. A refinement of the idea predicts the start of a sequential run of instructions given the address of the start of the previous sequential run of instructions.

This predictor reduces the recurrence above to:

- Hash the address of the first instruction in a run
- Fetch the prediction for the addresses of the targets of branches in that run of instructions
- Select the address corresponding to the branch predicted taken

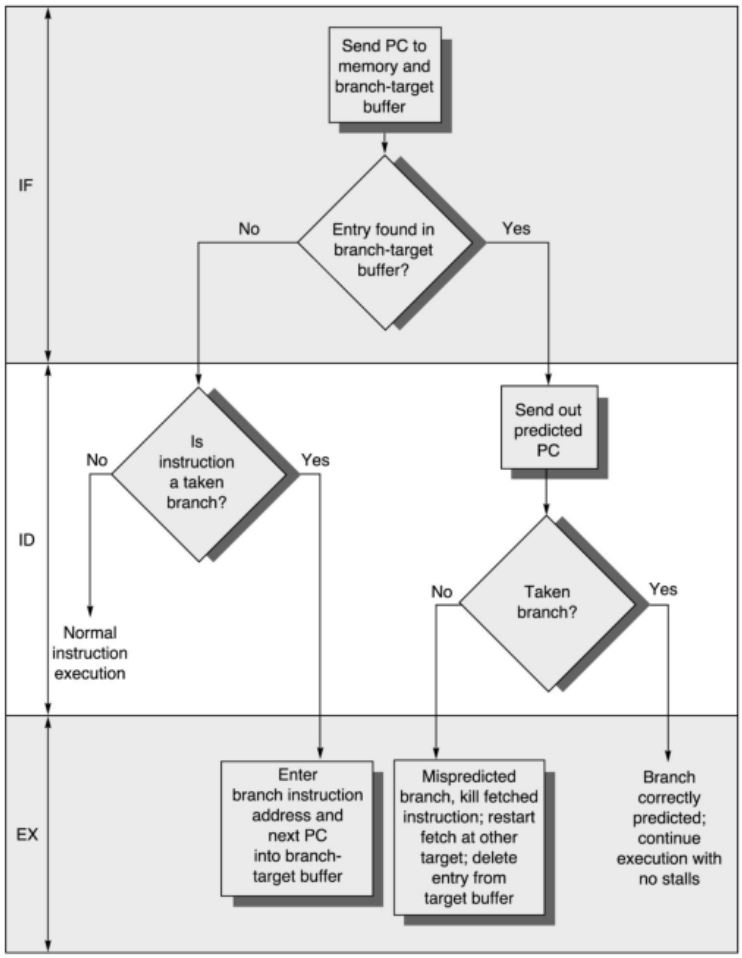
As the predictor RAM can be 5-10% of the size of the instruction cache, the fetch happens much faster than the instruction cache fetch, and so this recurrence is much faster. If it were not fast enough, it could be parallelized, by predicting target addresses of target branches.

To reduce the branch penalty further, we need to identify a branch and its predicted target in the **I** stage by using a **branch target buffer**.



The branch target buffer is a true cache, the full PC value must be compared to validate that this is a branch instruction before taking any action (we don't want to branch on an add instruction).

The target buffer scheme works as follows:



VLIW pipeline	Operation group allowed	Latency
1	Control, ALU	2
2	ALU, Load	3
3	Load / Store	3
4	Float	4
5	Float, Multiply	5